



New issue



RFC: advanced testing feature #2636

Open



lestofante opened on Feb 28, 2024



This is an RFC for improving the default testing system, this is based on personal experience and need, but try to remain kiss while providing helper functionality for testing on chip and on development machine. The following is to collect ideas and document what is present, and eventually expand upon it. All that I am describing can potentially be already implemented by someone skilled in rust build system and specific embassy build setup, the idea is to make this accessible (and documented) for everyone as simple as possible

We want to be able to test:

- application logic, or "business logic", probably a simple bunch of state machines
- application driver, probably driver for specific sensors and actuator

For the first part, we want a way to specify that a test is gonna be compiled and run on the development machine. Ideally no mocks are required, but:

- multiple test should be able to coexist
- a test may share file with other test (for example functionality to load recorded data and the actual recorded data)
- a test could require to include one or more file from the main application (unit and integration test)
- should be easy to call those test programmatically
- a test file may generate multiple test (to support conditional compilation)
- one or more test can be run from command line (just like normal tests)
- can be parallelized

Possible simple implementation with conditional configuration, the issue is to tell the build system to use a completely different compiler/project setup.

While those test could be placed anywhere, suggested to have them under path, relative to project root, "test/software/"

For application driver, our requirement we have all the above plus:

- a "companion" executable that will run on the developer machine; this companion executable will be given in input the debugger/communication interface. (*1)
- a "companion" executable that will run on a different target; this companion executable will be given in input the debugger/communication interface. (*2)
- one or more test can be run from command line (including programming to targets, may need to provide info like debug interfaces and target chips)
- a target chip can be a "visualized" chip

Again possible simple implementation with conditional configuration; the same trick to support different compiler/project setup could be reused to set up both companion executable.

While those test could be placed anywhere, suggested to have them under path, relative to project root, "test/hardware/" where the companion are the same name as the test file, but appending .verifier_sw.rs and .verifier_hw.rs

Possible issues:

- openocd should be able to support multiple client to the same target, but i wont be surprised if many clients breaks
- multi target compilation must be first class
- parallelization if of course limited by the amount of test hw connected to the computer unless virtualized

(*1) The idea is the executable is able to communicate with the chip, and even be able to halt/observe data trough the debugger.

Lets imagine I have a debugger and a serial connected to the chip, i should be able to send data trough the serial, and check if the receive buffer is in memory.

Of course this mean the debugger interface provide an easy API to add breakpoint, conditional breakpoint, halt, continue, read and write memory areas, depending on the debugger functionality.

(*2) there are test similar to (*1), but require specific timing, or HW capability. Technically this could be implemented with (*1); a second embedded project with multiple tests, the correct one get flesh in the second chip, and the script for that test return to the current test the debug interface, so now we have 2 debug interface, and can literally step the executions of the chips



jamesmunns on Feb 28, 2024

Contributor



Copy and pasting what I said in chat:

You describe a couple things here that range from "pretty straightforward" to "multi year R&D project".

I agree that embassy's setup w/ teleprobe targets a lot of the low hanging fruit already, especially wrt "do the drivers basically work".

Some of the other items, like "we can test anything on hardware" are... challenging! Especially when you need equally high fidelity inputs: what are the requirements you are testing? how do you specify the actual test cases you want to run?

These are all possible! But often I see:

- People who try to make it general and cover every case get "lost in the weeds", to cover everything would take infinite time and money
- People who cut down to their specific needs, like a test setup for their specific project, are usually fairly successful! But the results may not always transfer to a "general purpose" test suite, and it "feels like" a bunch of wasted/one-off effort

I have seen startups launch and fail on this topic, over the last 5-8 years alone. It's a real "white whale" of a topic for sure. I've even floated some ideas, but the long tail of work, met with "how do you sell/bill for this" is a hell of a calculation, and even more so in open source if nobody is getting paid.

I wrote about this years ago, of the kind of "flavors" of approaches I've seen to hardware testing:

<https://jamesmunns.com/blog/hardware-ci-overview/>

I'd probably consider embassy's CI setup to be a neat + effective combo of "host testing" and "HIL Testing (Easy)": It's running on-devices, but they devices are mostly tested via loopback.

There's some stuff that will miss (the sender and receiver are misconfigured in the same way), but pragmatically it's a great "cost:benefit" ratio.



lestofante on Mar 2, 2024

Author ...

Thanks for the answer.

As all the complex problem, best is to break it down to smaller byte size issue, and prioritize them.

TLDR:

To me, the most important is "Non-Host Testing", as if properly implemented will let the user be able to hack all the other system; we could then decide if implement other functionality right away as seen as helpful, or wait for community feedback on what is useful or not.

Too Short Wanna Read:

I personally developed and used a testing system with such capability for a critical system, my takeaway:

- Non-Host Testing:
by far the most used and useful feature even by interns, as most test included only a one or few sources, so it made easy to test a specific functionality and play around with it, with all the convenience of a native debugger and fast reload and executions.
The system started with small unit test when team build confidence, slowly grown into integration and even full PC simulation (same simulation later reused when we implemented HIL testing).
That made emerge state machine (by incentivizing proper code separation), keep in check nasty formulas and code path, general experimenting, and replay hours of recording in matter of a minutes.

"All" that the build system has to provide, is a way to tag the test as build on the target machine; creation of mocks, integration with simulators, how much or little to test, generation of chart with diff in the resulting data, integration with logic analyzer/oscilloscope (here we already talk of "hard" HIL Testing!).. is all left to the user, because as you said, everyone roll out their own solution, and there are great crates to do most of this stuff already.

The important is to give the base functionality; I think this is the point we should focus on. Rest will come naturally

-> the extra functionality provided by embassy integration would be to be able to tag a test to specify the target system

- Host Testing

On that note, again partial source testing was king.

we took only few classes/sources for each test; I found it very useful to test HAL in isolation and avoid regression, but also functionality that required strict timing or complex hw interaction. This again grown into full simulation on chip, where sensor where simulated by hardcoded or simulated data.

It could be implemented using Non-Host Testing, and from the test use probe-rs to upload the specific companion firmware for the test, and verify its execution.

So this would be really a set of helper function and tag to relate a test PC binary with a Target binary, build, upload with halted execution; and finally run the supervisor code passing the debug interface.

-> the extra functionality provided by embassy integration would be to be able to tag a software test with a firmware test, making sure they both get rebuild and are available, and the script that handle the upload and provide the probe handler

Not sure if "Simulated Host Testing" could be implemented for free by setting qemu as target, or would require a specialization of the script.

- Coprocessor

There are 2 variant

-- using a dedicated hardware like logic analyzer/signal analyzer that provide PC API interface: just use Non-Host Testing and call use API

-- self made coprocessor/dedicated coprocessor firmware for each test:

Again this is a something that could be implemented in Non-Host Testing, 2 instance of probe-rs are run, each upload its firmware. Up to the user supervisor code to manage breakpoint or result communication and inspection.

-> the extra functionality provided by embassy integration would be to be able to tag a software test with two (or more?) firmware test, and for each indicate what probe should be used.



[@lestofante](#) I am interested in this, but is the Embassy Issues the right place to have the discussion?

Perhaps a separate repo..?



lestofante on Nov 18, 2024

Author ...

[@lure23](#) i did already talk in chat, was suggested to open an issue.
Im still looking for this, but i have a few other project first to complete before this became my priority again



SL-RU on Jan 19

...

We can look at the nuttx, it has very advanced testing tooling



Add a comment

H B I | ≡ <> 🔗 | ≡ ≡ ≡ | @ ↗ ↶ 📎

Write Preview

Use Markdown to format your comment



Paste, drop, or click to add files

Close issue



Comment

Metadata

Assignees

No one assigned

Labels

No labels

Type

No type

Projects

No projects

Milestone

No milestone

Relationships

None yet

Development

No branches or pull requests

Notifications

[Customize](#)

 **Subscribe**

You're not receiving notifications from this thread.

Participants

